Reimplementation of DETR
A Unified Transformer-based Framework for Object Detection

Anonymous submission

Abstract

Traditionally, the object detection task has been performed by specialized CNN networks, like RCNN, Fast RCNN, and Faster RCNN. However, these neural networks require multi-stage training and hand-created features, like non-maximal suppression, to work properly. Thus, in 2020, Facebook AI proposed solving the problem of object detection with a modified vision transformer model that was simple in its architecture, could be trained all at once, and required no hand-crafted features. Their model was named DETR(DEtection Transformer). This model led to the creation of many other DETR-like models, including Deformable DETR, DAB DETR, DN DETR, DINO, and Mask-DINO, with DINO and Mask-DINO, with the DINO and Mask-DINO models achieving state-of-the-art results in object detection, and with Mask-DINO achieving state-of-the-art results in panoptic, semantic and instance segmentation, proving the validity of using vision transformers instead of convolution neural networks for the task of object detection. Thus, the goal of this project is to reimplement this pivotal model from scratch and achieve the same average precision result as the 2017 COCO Object Detection validation dataset. The model architecture was replicated and then trained on the 2017 COCO Object Detection training dataset for 30 epochs. Then, the model was evaluated using mAP(for IOU from 0.5-0.95) on the validation set. The reimplemented model achieved significantly worse mAP than the DETR model, primarily due to the lack of GPU computing available. The original DETR model was trained on 16 Nvidia V100 GPUs for over 300 epochs, training for over 3 days, which cost around 4000 USD. The reimplemented version of DETR trained on a mix of 1 A100, L4, or T4 Nvidia GPU for 30 epochs, training for 5 days, which cost around 80 USD. The discrepancy in performance is primarily due to the difference in training epochs. The code and demo video can be found here: https://anonymous.4open.science/r/DETR_Reimplementation-B675

Introduction

The topic of this project is to use vision transformers to create a universal object detection model. This branch of the domain of computer vision began with the creation of the transformer model(Vaswani 2017) in 2017. However, it was not until 2020 that the transformer, created for processing language, was applied to images, terming this new type of transformer as a Vision Transformer or a ViT(Dosovitskiy et al. 2021).

The primary reason that it took three years to make the jump from utilizing transformers for their initial domain of natural language processing to the domain of image processing, firstly through the task of image classification, is that transformers require much more data to train than convolutional neural networks, like BiT(Kolesnikov et al. 2020), which previously held the state-of-the-art performance on tasks like image classification, before the introduction of vision transformers in 2020. However, while vision transformers generally require more time and data to train, their appeal is that they are a much simpler architecture than, for instance, BiT. Vision transformers can also be trained end-to-end(all learned parameters can be trained at the same time), while specialized CNN networks, like RCNN, Fast RCNN, and Faster RCNN(Ren et al. 2016), require parts of the architecture to be trained separately, so they introduce more overhead into the training process.

However, the main benefit of the vision transformer is that its simplistic models(in comparison to CNN's) created based on a transformer architecture can better generalize to different domains or a much broader dataset, as they do not have the in-built inductive biases of convolutional neural networks. So, they must learn when to apply those biases and when not to. The inductive biases referred to are concepts such as locality and translational invariance. Locality is the fact that if something wants to determine what an object is or the properties of that object, it is more important to look at the pixels around that object than the pixels on the other side of the image. Translational invariance is the property that if the location of an object shifts in the image, it is still the same object, and the model should determine it as such.

These two properties are built into the architecture of the convolutional neural network, whereas a vision transformer must learn these properties, which is one reason why vision transformers take longer to train; however, it means that a vision transformer can learn when it is important to not follow the principle of locality, by changing where it places attention(in the multi-head attention layer of the transformer) in the image as it also takes as input the whole image, instead of just a small portion as for a convolutional neural network.

Thus, this generalizability led swiftly to the creation of DETR(Carion et al. 2020), which is an end-to-end object detector that was proven to be capable of generalizing to

perform panoptic segmentation(where a pixel-level mask is placed over each object in the image identifying what type of object it is and giving it a unique mask) as well. The importance of DETR and the DETR-like models that followed is that they are all tackling the problem of creating a single universal object detector using variations of the vision transformer architecture. Object detection and segmentation(which will be discussed later) are vital for developing almost all autonomous systems that interact with the physical world, as it is the manner in which any system will determine how to navigate through the world or properly manipulate objects in the world. The main applications are in the fields of healthcare, defense, manufacturing, and transportation.

Thus, the objective of this project is to reimplement the transformative model of DETR from scratch and achieve similar results to the original DETR model on the COCO object detection dataset to learn how vision transformers can be applied to the problem of object detection as its successors(which enhanced and extended the simple transformer model of DETR) they currently hold state-of-the-art performance in this very important domain, and so it is essential that I understand the fundamental operation of DETR-like models, so that I can understand how to apply them to the problems and fields listed above.

Related Work

As stated previously, the main model being examined through this project is the DETR model. DETR is the foundation of a class of models(referred to as DETR-like models) that extended the original DETR model so that this class of models could eventually achieve state-of-the-art performance in object detection. The main contributions to the class of DETR-like models are Deformable DETR(Zhu et al. 2020), Dynamic Anchor Box DETR(Liu et al. 2022), Denoising DETR(Li et al. 2022), DINO(Zhang et al. 2022), and Mask DINO(Li et al. 2023). The two primary other works that will be discussed in this section are DINO and Mask DINO. DINO(DETR with improved denoising anchor boxes) will be discussed because it combined all of the enhancements and extensions made in Deformable DETR, DAB DETR, and DN DETR to achieve state-of-the-art performance on the task of object detection. Mask DINO will be discussed because it added a "mask" component to DINO that allowed it to achieve state-of-the-art performance not only in object detection but also in image segmentation.

First, to provide context to the explanations of DINO and Mask DINO the DETR model will be discussed. As can be seen in Figure 1, the architecture for DETR is that of a basic transformer, and there are not many variations compared to the original vision transformer.

The main variation of the DETR model is that it uses a convolutional neural network backbone(ResNet50) to create a set of image features that are then inputted into a standard transformer encoder. The other main difference in the architecture of DETR is that the input to the transformer decoder is a set of object queries. These object queries are a set of vectors that instantiate which part of the image the transformer should focus its attention on and with how much in-

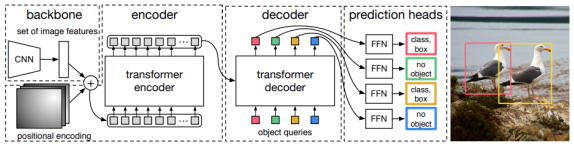

Figure 1: Illustration of the full architecture of the DETR model.

tensity it should focus its intensity. When transformed into a two-dimensional matrix, they show where in the input image(center, top left, right, bottom) that specific query wants to focus on. These object queries are then inputted as the values, keys, and queries to the multi-head self-attention layer of the decoder and to the queries of the multi-head cross-attention layer of the decoder. Then, the output of the decoder is a set of processed object queries, which, when passed through a feed-forward network, results in a class and bounding box.

The other major contribution of the DETR model is its loss function. For calculating the loss during training, the DETR model performs a maximal matching(where if one more edge would be added to the graph, it would no longer be matching) of a bipartite graph, in which a set of edges is chosen so that no two edges share the same node between the two sets of nodes present. Each set of nodes is made up of N nodes(where N is the maximum number of bounding boxes that can be created). One set is the predicted bounding boxes by the model, and the other set is the true set of bounding boxes(either containing an object or no object). The loss is determined by performing matching between the two sets and then subtracting based on how much each matched box overlaps and if the matchings are of the correct class(like a no object box to a no object box).

The significance of this bipartite graph-based loss function is that it drastically reduced the number of duplicate boxes as the model was rewarded for having exactly the correct number of objects and no object boxes as were in the image. Thus, this implicit elimination of duplicates made the use of non-max suppression(a common technique for other object detection models at the time to get rid of duplicates by only choosing the most probably bounding box in a group of bounding boxes) unnecessary, which is good as non-max suppression generally makes a model worse at detecting multiple bounding boxes close to each other as it thinks some are duplicates.

Additionally, the major result of the DETR paper is that it was able to match the Faster RCNN(a previous state-of-the-art model) model in all AP metrics on the COCO object detection dataset with the same number of parameters, proving that vision transformers are a viable route to creating state-of-the-art transformer-based universal object detectors.

The strengths of this paper are that the model architecture they used is a simple transformer with only slight modifications to make it function for the task of object detection. The simplicity of the architecture makes the model good at generalizing to other tasks, like panoptic image segmentation, which the authors qualitatively and quantitatively show

the model is capable of doing through some sample output panoptic segmentations produced from the model. Additionally, to create these segmentations, the DETR model was trained to detect object and things boxes instead of just object boxes. Also, a panoptic head was added onto the model that took as input the box encodings produced by the transformer decoder and predicted a binary mask for each of the predicted boxes by first creating new attention maps by passing the boxes through a multi-head attention layer; then the attention maps are converted into masks by passing them through an FPN-style convolutional neural network. To acquire the final output, a pixel-wise argmax is performed to determine which class each pixel is most likely to be.

The other main strength of the DETR model is that it has a stronger performance in detecting large objects than the Faster R-CNN model, which is likely due to it(as a transformer) being able to consider the whole image when considering where to place attention, whereas CNN's, through their implicit property of locality, have greater challenges linking distant parts of an image together.

However, one of the main challenges the DETR model faces is it struggles more with detecting smaller images as, unlike CNN's, it does not implicitly focus on local areas, and so, this is an area where using a simple transformer architecture makes the model struggle. Additionally, while it can perform panoptic segmentation, its performance, in comparison to object detection, drops noticeably, showing that it is not good at generalizing to other tasks besides object detection, which is one of its main gaps that would need to be addressed for the creation of a universal object detector and segment.

Next, the DINO model improves the object detection architecture of DETR primarily by adding three key components: mixed query selection, look forward twice, and contrastive denoising loss.

First, mixed query selection essentially combines anchor boxes generated from the output of the encoder once they have been passed through a linear layer(called dynamic anchor boxes, as they are manually set beforehand) with the traditional trainable query inputs from DETR. These dynamic anchor boxes help improve object detection when the objects are all in close proximity to each other as it allows each anchor box(depending on its orientation and what part of the image it includes) to "look for" a specific type of bounding box. Thus, if two objects are in the same place, two different anchor boxes with different orientations or shapes would pick up each of the objects, so they would both be detected. Additionally, anchor boxes are like bounding boxes as the bounding boxes are computed based on their relation to one of the anchor boxes in the set of them(which one they are most similar to).

Second, look forward twice is the method in which the decoder layers of the transformer are connected. It is not explained well in the literature of DETR-like models, but the complex gradient calculations needed to backpropagate through all the successive attention and other layers present are unstable, so the original method devised to solve this problem was the look forward once method. In this method, the prediction at a certain layer was only based on input to

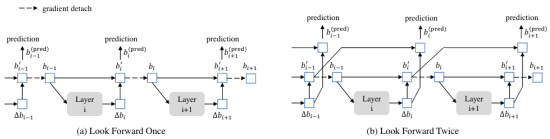

Fig. 6. Comparison of box update in Deformable DETR and our method.

Figure 2: Illustration of the difference between the look forward once and look forward twice models.

the previous layer and the output of the previous layer. Then, the result was copied(but not propagated) to the next layer, and the cycle was repeated; thus, the gradient was only propagated for the inputs and outputs of one layer at a time, fixing the instability problem. However, the DINO model implements the look forward twice method for gradient propagation, in which the anchors(input and output) of the current layer are connected to the gradient of the current layer(like in look forward once) but also to the gradient of the next layer. They do not explain much more about the method or why a look forward three times or four times would not work better. However, they proved with an ablation study that looking forward twice produces better, more accurate bounding boxes than the look forward once method. A diagram of each method can be seen in Figure 2.

Thirdly, contrastive denoising loss is where, besides the N queries(anchor boxes + trainable vectors) inputted into the decoder, there are also ground-truth bounding boxes and very noisy ground-truth bounding boxes inputted to the decoder. Then, the output from those inputs needs to be matched to the original ground-truth box for the noisy case and the no-object box for the very noisy case. This method teaches the model more concretely when to say yes to an anchor box and when to reject an anchor box that is not close to any object. Thus, this method, which is only implemented in the model during training, helps improve the time to converge and the test performance of the model.

Overall, the DINO paper demonstrates how transformer-based object detectors can achieve state-of-the-art performance in a clear and succinct manner. It also shows through ablation studies that the three changes described above significantly improve the performance of the model(measured in AP) in comparison to the previous extended versions of the DETR model(Deformable DETR, DAB DETR, and DN DETR), emphasizing the significance of their work. Additionally, another strength of their model is that it is much more GPU compute efficient when using a smaller backbone(ResNet 50) in comparison to other DETR-like models and RCNN. They also show that when using its largest backbone(Swin-L) it is still much more efficient than other large state-of-the-art models as it uses a fraction of the parameters and pre-training dataset size to achieve better results.

To preface the explanation of Mask-DINO, it extends DINO to the image segmentation task, which assigns each pixel in an image to a specific class at different levels. For instance, instance segmentation is where a unique mask is

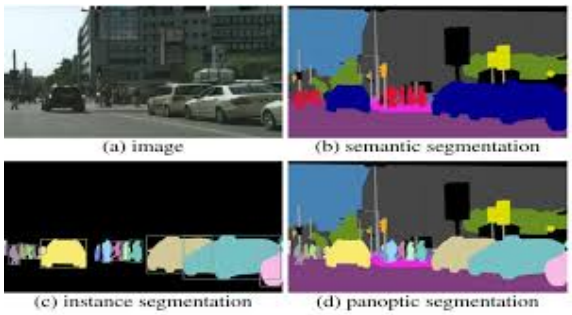

Figure 3: Illustration of all panoptic, instance, and semantic segmentation in reference to a source image.

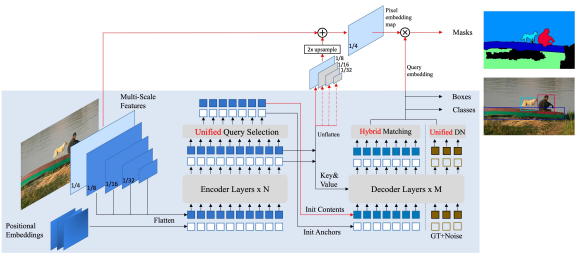

Figure 4: Illustration of the DINO model architecture, and the modifications Mask-DINO makes(in red).

placed over each object only, whereas semantic segmentation is where a unique mask is placed over each type of object and thing. In contrast, panoptic segmentation is where a unique mask is placed over every object and thing in an image. Refer to Figure 3 for an illustration of the different types of segmentation.

Mask-DINO maintains this architecture from DINO but adds a couple of features, both of which can be seen in Figure 4. To preface, most of the features added to the DINO model to form the Mask DINO model are from the Mask2Former model(Cheng et al. 2022), which was a transformer model published by Meta AI. The model was designed only for image segmentation, and it achieved state-of-the-art performance at all levels(panoptic, instance, and semantic). First, Mask-DINO modifies mixed query selection by outputting initial masks and dot-producing the predicted initial anchor boxes with high-resolution feature maps to create the predicted initial masks. They also modify the contrastive denoising loss by helping to improve the convergence of the mask prediction side of the model by inputting noised ground-truth bounding boxes into the mask predictor and getting out their correlated masks, as boxes are essentially noised masks. Finally, for predicting the masks, they add a completely new branch to the model(DINO), which combines upsampled outputs from the encoder with multi-scale features from the input image and then combines them with query embeddings from the output of the decoder(matched embeddings for each object/thing in the image) to create the masks(very similar to the process Mask2Former takes).

One of Mask-DINO's strengths is that it leverages the extra information from the multi-scale features and predicted masks to make it more generalizable than DINO. Mask-DINO is more generalizable as the more fine-grained information from the masks enhances the bounding box predictions, and the bounding box predictions(enhanced by the use of the modified contrastive denoising loss) enhance the predictions of the masks. The authors of the paper do a good job of laying out how the properties of the model and the interactions between the box and mask prediction portions of the model enhance the overall performance of the model and make it more generalizable.

Additionally, this enhanced performance is demonstrated by the Mask-DINO model achieving state-of-the-art performance in the object detection task on the COCO dataset and state-of-the-art performance in panoptic, semantic, and instance segmentation on the Object365 dataset. Therefore, it proves the validity of using a vision transformer to create a universal model for all detection and segmentation tasks.

However, while the Mask-DINO model is good, its main limitations are that it, like Mask2Former, struggles to achieve the same performance on one of the segmentation tasks when it is only trained on two of three(like testing on panoptic segmentation when only trained on semantic and instance segmentation). Additionally, in the large-scale setting, they did not set a new SOTA among these models because the segmentation head requires more GPU memory, and so they have to accommodate for this increase in GPU memory from the segmentation head with a decrease in the number of input queries, in comparison to the DINO model, which hurts the performance of the model. Thus, in the future, the model needs to be made more compute-efficient to accommodate the addition of the segmentation head.

Problem Definition

The primary paper that I selected to reimplement is DETR. This paper tries to create a universal end-to-end object detection model. The purpose of this model was to prove that simple vision transformers can achieve better performance than specialized convolutional neural networks with handcrafted features. While it did not achieve state-of-the-art performance, it was the foundation of the other two selected papers, DINO and Mask DINO. DINO achieves state-of-the-art performance on object detection, and Mask DINO achieves state-of-the-art performance on object detection and image segmentation at all levels(panoptic, semantic, and instance).

Thus, to fully reimplement the DETR model, Pytorch must be used to create the vision transformer(encoder and decoder layers), train it on the COCO 2017 Object Detection training dataset, evaluate it on the COCO 2017 Object Detection validation dataset, and compare its mAP to that reported by the authors of the DETR model to assess how well the DETR model was reimplemented.

Methodology

Since the DETR model requires Nvidia GPUs to train, and the only way I can access Nvidia GPUs is through Google Colab, the whole implementation was written in a

Jupyter Notebook in Google Colab. The notebook was divided into six primary sections: preliminaries, dependencies, pre-processing, architecture, training, and testing. Each section was based on the main tasks needed to fully reimplement the DETR model. For the preliminaries section, the main action taken was to mount the Google Drive to Google Collab so that checkpoints(containing trained model weights and the loss history) could be saved during the training process and loaded for the testing process.

Then, for the dependencies section, all necessary libraries(pytorch, COCO, torchvision, numpy, matplotlib) were imported. Also, the 2017 COCO training and validation datasets, as well as their corresponding annotations, were downloaded.

The pre-processing section was then composed of a script that defines all of the 91(90 objects + 1 for "no object") classes in the COCO dataset. It also defines functions for plotting images with bounding boxes of all detected objects laid on top of the image with the corresponding class linked to the bounding box detection. It also defines code to convert the bounding boxes in the training and validation data from the COCO format to the format the DETR model expects, filters out crowded annotations as defined in the COCO dataset, and filters out invalid bounding boxes and scales the bounding boxes up relative to the image size(from the normalized 0 to 1 scale). This code is part of an extension of the standard COCO detection data processor, which also normalizes the image data and then resizes and returns the data(boxes, classes, and images) in a format expected by the DETR model. The last part of this section is the collatefn, which is used as part of the training and validation data loader for formatting batched data by stacking the images along the batch dimension, and gathering the class labels and bounding boxes into tuples. This format is necessary for models like DETR, which process multiple instances per image in a single batch.

The architecture section is composed solely of the Pytorch implementation of the DETR model. The module first loads the ResNet50 model as its feature extractor(as it is the lightest-weight backbone used in the paper). It also defines a one-by-one convolutional layer to transform the output from the ResNet50 backbone into the input dimensions needed by the transformer. Then, it initializes the positional encodings as trainable weights, as it was shown in the paper, to produce equivalent results and more straightforward to implement than fixed positional encodings(more akin to those implemented in the original 2017 transformer paper). Then, the encoder and decoder are defined separately using the standard pytorch implementations, with both the encoder and decoder consisting of six encoder and decoder layers and 0.1 dropout(as that is what the DETR paper selects). Finally, two linear layers are defined, one to convert the output of the decoder into class predictions and one to convert the output of the transformer into bounding box predictions.

The training section then consists of three main scripts: a definition of the loss function, a script to initialize the training parameters and the primary training script. The loss function takes in a singular predicted bounding box, class distribution, and the corresponding ground truth box and class. First, Hungarian matching is performed(through the linear sum assignment function in the scipy module), which is a type of bipartite matching that finds the optimal matching of all the bounding boxes and class distributions predicted by the DETR model for an image with the ground truth bounding box and class labels by optimizing a cost matrix which contains the sum of the generalized intersection over the union of the ground truth and predicted bounding boxes, as well as, the L1-norm of the difference between respective bounding boxes, and the L1-norm between the box class probabilities. Then, once the optimal matching is determined, the loss function returns three loss values: the generalized intersection over the union, the L1-norm of the bounding boxes, and the cross entropy loss of the class labels of the optimal matching.

The next part of the training section is a script that initializes the DETR module and then initializes the Adam optimizer with a learning rate of 0.00001 for the backbone parameters and a learning rate of 0.0001 for every other parameter(based on learning rates set in the DETR paper). Then, the primary training script loads the images into batches of thirty-two and computes the loss for each image in the batch using the function described above(it sums the three losses output by the loss function). Then, it averages the loss over the batch and performs backpropagation. It then does this for each batch in the epoch, and then at the end of each epoch, it saves the weights and loss history(array containing the model loss at each epoch). Then, after training is complete, the training loss vs epoch is plotted.

The last section of the implementation, the testing section, consists of two main parts: evaluating the trained DETR model on the COCO validation dataset and performing inference by plotting ground truth vs. predicted output bounding boxes on input validation images. First, the DETR is evaluated on the COCO validation dataset using the mean average precision metric, which in this context means the mean of the average precision values at ten equally spaced intersections over union thresholds from 0.5 to 0.95(to match the thresholds used to calculate mAP in the original DETR implementation). The mean average precision is calculated at each threshold by first converting the ground truth and predicted bounding boxes to the format COCO expects; it then sorts the bounding box and class prediction pairs by the model confidence that the prediction is correct(softmax probability). Then it calculates the intersection over the union for each predicted and ground truth bounding box, and then it uses the computed intersection over the union to find the best matching for the ground truth and predicts bounding boxes while also determining the number of true positive and false positive results, which is used to compute the average precision for this image. That calculation is then repeated for each image in each batch and then averaged over each batch, and then the average over all batches is calculated and returned.

The primary evaluation script consists of applying the sigmoid to the output of the DETR model that produces bounding box predictions and applying the softmax function to the output of the model that produces class label predictions. Then, arrays of these outputs put through the sigmoid and

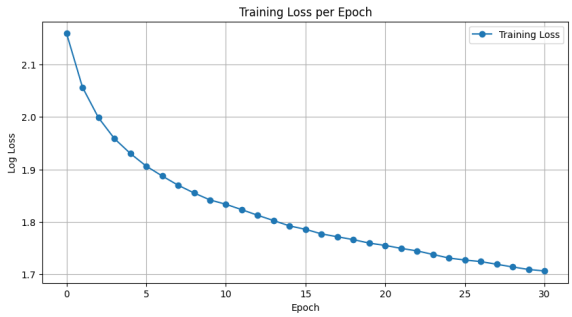

Figure 5: Plot of the training log loss for each epoch of training.

softmax functions, along with their corresponding ground truth data, are inputted into the function described above to calculate the mean average precision.

The final part of the testing section is the inference script, which takes a batch(two images) from the evaluation dataset, inputs them into the DETR model, takes the output, applies the softmax and sigmoid as described above, and then gets the argmax(for predicted class) and max(for confidence in prediction) of the class prediction distribution. It then outputs the predicted vs. ground truth bounding boxes, classes, and confidences for each image.

Experimental Results and Result Analysis

After training the model for thirty epochs on a mix of an A100, L4, and T4 GPU for five days(at the cost of 80 USD in Google Colab), the reimplemented version of DETR achieved an mAP of 1.6 on the 2017 COCO object detection validation set. The original DETR model which trained on 16 Nvidia V100 GPUs for over 300 epochs, training for over 3 days, which cost around 4000 USD, achieved an mAP of 42.0 on the COCO object detection validation set. The primary reason for the discrepancy of 39.4 mAP between the performance of the reimplemented and original versions of DETR is training epochs. Based on the graph in Figure 5, which shows the training loss vs. number of epochs trained for, the loss had not flattened out and would continue to decrease as it was trained for more epochs. Additionally, the reason that the mAP is so low, even though it had already trained for thirty epochs, is that, as can be seen in the output of model inference in Figure 6, the outputs of the model either do not have any bounding boxes or very few that only cover parts of the objects, which I believe is due to the attention mechanism being in the transformer of the DETR model looking at the whole image and trying to find relation weights for every single pixel in the image equally, which means that it takes a very long time for the model to learn general features and objects to detect. The addition of no object classes to match, which is important for reducing duplicate boxes, I believe, also biases the network away from being overly confident in box predictions, meaning it would take longer to determine where to predict an object's bounding box.

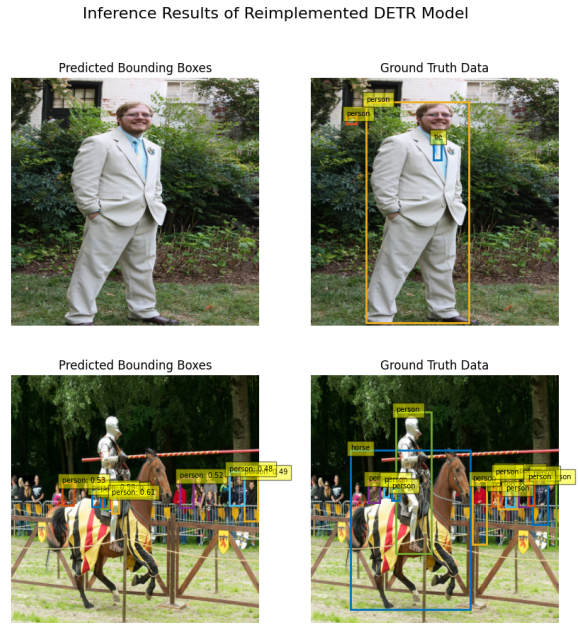

Figure 6: Illustration of the model output for two testing images.

Conclusion and Future Direction

Overall, the reimplemented DETR model replicated the architecture of the pivotal DETR for object detection, and with more epochs of training, it would replicate the results of the original model. The main constraint of the reimplemented model was that I did not have enough money to train the model for the full around three hundred epochs(the number of epochs the original DETR was trained for) as if given thirty epochs of training cost 80 USD; the full training would cost around 800 USD. However, given the log-loss curve in Figure 5, I am confident that given sufficient funds and so sufficient training epochs, the reimplemented DETR would match the results of the original model.

Given the conclusion made above, the first potential future direction for this project is to train for at least three hundred epochs and then re-evaluate the model's performance to confirm it matches the original DETR's performance. The reimplemented DETR model could also be trained for much longer than three hundred epochs(for instance, five hundred epochs) to see if it overcomes the flatline it reaches at around three hundred epochs in the original DETR paper and finds a better minimum and so achieves a better mAP on the COCO object detection validation dataset.

The other possible future direction would be to add a segmentation head to the model so that the DETR model can perform image segmentation and object detection. This process is described briefly in the original DETR paper, so it would be good possible future direction to add this head to the reimplemented DETR model. The head could either be added separately (as is done in the original paper), where the DETR model can only perform object detection or image segmentation separately, or the DETR could be modi-

fied in a manner similar to how the DINO model was modified to create Mask DINO, to make a Mask DETR model that is capable of performing object detection and image segmentation(panoptic, semantic, and instance) simultaneously. I think the latter option especially would be a good future direction as it would extend the work done in the original DETR paper and be a more proper demonstration of the power of DETR to generalize to image segmentation than the demonstration shown in the original DETR paper.

References

Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; and Zagoruyko, S. 2020. End-to-end object detection with transformers. In *European conference on computer vision*.

Cheng, B.; Misra, I.; Schwing, A. G.; Kirillov, A.; and Girdhar, R. 2022. Masked-attention mask transformer for universal image segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*.

Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; Uszkoreit, J.; and Houlsby, N. 2021. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. In *International Conference on Learning Representations*.

Kolesnikov, A.; Beyer, L.; Zhai, X.; Puigcerver, J.; Yung, J.; Gelly, S.; and Houlsby, N. 2020. Big transfer (bit): General visual representation learning. In *Computer Vision–ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part V 16*, 491–507. Springer.

Li, F.; Zhang, H.; Liu, S.; Guo, J.; Ni, L. M.; and Zhang, L. 2022. Dn-detr: Accelerate detr training by introducing query denoising. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 13619–13627.

Li, F.; Zhang, H.; Xu, H.; Liu, S.; Zhang, L.; Ni, L. M.; and Shum, H.-Y. 2023. Mask dino: Towards a unified transformer-based framework for object detection and segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*.

Liu, S.; Li, F.; Zhang, H.; Yang, X.; Qi, X.; Su, H.; Zhu, J.; and Zhang, L. 2022. Dab-detr: Dynamic anchor boxes are better queries for detr. *arXiv preprint arXiv:2201.12329*.

Ren, S.; He, K.; Girshick, R.; and Sun, J. 2016. Faster R-CNN: Towards real-time object detection with region proposal networks. *IEEE transactions on pattern analysis and machine intelligence*, 39(6): 1137–1149.

Vaswani, A. 2017. Attention is all you need. *Advances in Neural Information Processing Systems*.

Zhang, H.; Li, F.; Liu, S.; Zhang, L.; Su, H.; Zhu, J.; Ni, L. M.; and Shum, H.-Y. 2022. Dino: Detr with improved denoising anchor boxes for end-to-end object detection. *arXiv preprint arXiv:2203.03605*.

Zhu, X.; Su, W.; Lu, L.; Li, B.; Wang, X.; and Dai, J. 2020. Deformable detr: Deformable transformers for end-to-end object detection. *arXiv preprint arXiv:2010.04159*.